부록 · Docker 로 DB 설치

Docker PostgreSQL 설치 교본

이 문서에서 고친 곳

- \gx 뒤의 세미콜론을 뺐습니다. \gx 자체가 끝 표시라 붙이면 「invalid command \gx;」 오류가 납니다(실측).
- pg_dump 를 받을 때 docker exec -t 를 뺐습니다. -t(터미널)를 붙이면 줄 끝이 CRLF 로 바뀌어 덤프 파일이 오염됩니다 (실측: CR 41개 → 0개).

Rocky Linux + Docker Compose · 초보자용

목표: PostgreSQL을 Docker Compose로 설치하고 설정파일 분리, 데이터 영구저장, devuser 계정, 외부 5432 접속, DBeaver 연동까지 구성함.

1. 최종 구성

```text
/home/devuser/docker/postgresql/
├── compose.yml
├── .env
├── .psqlrc
├── config/
│   ├── postgresql.conf
│   └── pg_hba.conf
├── init/
│   └── 01-init.sql
└── data/                  ← 실제 PostgreSQL 데이터 영구 저장
```

가장 중요한 디렉터리는 ~/docker/postgresql/data 임. 컨테이너를 삭제하거나 재생성해도 이 폴더를 삭제하지 않으면 DB 데이터가 유지됨.

2. Docker 설치 여부 확인

```bash
docker --version
docker compose version
sudo systemctl status docker
```

```bash
sudo systemctl enable --now docker
```

3. 기존 PostgreSQL 및 5432 포트 확인

```bash
rpm -qa | grep -Ei 'postgresql|postgres'
sudo ss -lntp | grep 5432
```

직접 설치된 PostgreSQL을 제거하려면 필요한 DB를 먼저 백업해야 함. 5432가 비어 있어야 Docker PostgreSQL이 기본 포트를 사용할 수 있음.

4. 디렉터리 생성

```bash
mkdir -p ~/docker/postgresql/{data,config,init}
cd ~/docker/postgresql
pwd
ls -la
```

5. .env 환경설정

```bash
nano .env
```

```env
POSTGRES_DB=wizdb
POSTGRES_USER=devuser
POSTGRES_PASSWORD=본인이정한강력한비밀번호
POSTGRES_PORT=5432
```

```bash
chmod 600 .env
```

POSTGRES_USER와 POSTGRES_DB는 빈 data 디렉터리에서 최초 초기화할 때 생성됨.

6. postgresql.conf 만들기

```bash
nano config/postgresql.conf
```

```text
listen_addresses = '*'
port = 5432

max_connections = 100
shared_buffers = 256MB
work_mem = 4MB
maintenance_work_mem = 64MB

logging_collector = on
log_directory = 'log'
log_filename = 'postgresql-%Y-%m-%d.log'

timezone = 'Asia/Seoul'
log_timezone = 'Asia/Seoul'

password_encryption = 'scram-sha-256'
```

listen_addresses='*' 는 모든 네트워크 인터페이스에서 연결을 받을 수 있게 함. 실제 허용 IP는 pg_hba.conf와 방화벽에서 제한하는 것이 안전함.

7. pg_hba.conf 만들기

```bash
nano config/pg_hba.conf
```

```
local   all       all                     trust                                   text
host    all       all   127.0.0.1/32        scram-sha-256
host    all       all   ::1/128             scram-sha-256

# 내부망에서만 외부 접속 허용
host    all       all   192.168.0.0/24      scram-sha-256
```

0.0.0.0/0 으로 허용할 수도 있지만 인터넷 전체를 인증 대상으로 만들 수 있으므로 내부망 사용 시 192.168.0.0/24처럼 제한하는 것을 권장함.

8. 최초 초기화 SQL

```
nano init/01-init.sql                                                           bash
```

```
CREATE SCHEMA IF NOT EXISTS wiz;                                                sql

CREATE TABLE IF NOT EXISTS wiz.member (
    id BIGSERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(200),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

docker-entrypoint-initdb.d 아래의 SQL은 data 디렉터리가 비어 있는 최초 초기화 때만 자동 실행됨.

9. psql 편의 설정

```
nano .psqlrc                                                                    bash
```

```
\x auto                                                                         sql
\pset null '(NULL)'
\timing on
```

\x auto는 결과가 화면 폭보다 길 때 자동으로 세로 확장 출력함. 항상 세로는 \x on, 항상 가로는 \x off 임.

10. compose.yml 전체 설정

```
nano compose.yml                                                                bash
```

```yaml
services:
  postgres:
    image: postgres:17
    container_name: wiz-postgresql
    restart: unless-stopped

    env_file:
      - .env

    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}

    ports:
      - "${POSTGRES_PORT}:5432"

    volumes:
      - ./data:/var/lib/postgresql/data:Z
      - ./config/postgresql.conf:/etc/postgresql/postgresql.conf:ro,Z
      - ./config/pg_hba.conf:/etc/postgresql/pg_hba.conf:ro,Z
      - ./init:/docker-entrypoint-initdb.d:ro,Z
      - ./.psqlrc:/root/.psqlrc:ro,Z

    command:
      - postgres
      - -c
      - config_file=/etc/postgresql/postgresql.conf
      - -c
      - hba_file=/etc/postgresql/pg_hba.conf

    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 10s
      timeout: 5s
      retries: 5

    networks:
      - db-network

networks:
  db-network:
    driver: bridge
```

Rocky Linux SELinux 환경을 고려하여 bind mount에 :Z 옵션을 사용함. YAML은 TAB이 아니라 공백으로 들여쓰기해야 함.

11. Compose 문법 검사 및 시작

```bash
docker compose config
docker compose up -d
docker ps
```

```bash
docker compose config에서 YAML 오류가 없는 것을 확인한 뒤 up -d를 실행함.
```

12. 로그 및 5432 확인

```bash
docker logs wiz-postgresql
sudo ss -lntp | grep 5432
```

정상이라면 0.0.0.0:5432 또는 [::]:5432에 docker-proxy가 LISTEN 중인 것을 확인할 수 있음.

13. devuser으로 PostgreSQL 접속

```bash
docker exec -it wiz-postgresql psql -U devuser -d wizdb
```

정상 접속되면 wizdb=# 또는 wizdb=> 프롬프트가 표시됨.

```sql
SELECT current_user, current_database();
\l
\du
\dn
\dt
\q
```

14. users 예제 테이블

```sql
CREATE TABLE users (
    id BIGSERIAL PRIMARY KEY,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(150) UNIQUE,
    phone VARCHAR(20),
    role VARCHAR(20) DEFAULT 'USER',
    enabled BOOLEAN DEFAULT TRUE,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

```sql
INSERT INTO users
(username, password, name, email, phone, role)
VALUES
('hong', '1234', '홍길동', 'hong@test.com', '010-3333-4444', 'USER');

SELECT * FROM users;
SELECT * FROM users \gx
```

실제 로그인 시스템에서는 평문 비밀번호를 저장하지 않고 BCrypt 등 안전한 해시 방식을 사용해야 함.

15. 테이블과 구조 확인

```sql
\dt
\d users
SELECT * FROM users;
```

psql에서 \x on은 계속 세로 출력, \x off는 가로 출력, \x auto는 자동 선택, SELECT ... \gx는 해당 조회만 세로 출력함.

16. Rocky Linux 방화벽

내부망 192.168.0.0/24에서만 5432 접속을 허용하는 예임.

```bash
sudo firewall-cmd --permanent \
  --add-rich-rule='rule family="ipv4" source address="192.168.0.0/24" port port="5432" protocol="tcp" accept'

sudo firewall-cmd --reload
sudo firewall-cmd --list-all
```

17. Windows에서 포트 테스트

현재 Rocky Linux 서버 내부 IP가 SERVER_IP인 환경 기준:

```powershell
Test-NetConnection SERVER_IP -Port 5432
```

TcpTestSucceeded : True가 나오면 Windows PC에서 Docker PostgreSQL 포트까지 네트워크 연결이 성공한 것임.

18. DBeaver 연결

항목	값
DB 종류	PostgreSQL
Host	SERVER_IP
Port	5432
Database	wizdb
Username	devuser
Password	.env에 설정한 비밀번호

```text
jdbc:postgresql://SERVER_IP:5432/wizdb
```

19. devuser 권한 확인 및 관리

```sql
\du
SELECT current_user;
```

```env
POSTGRES_USER=devuser으로 최초 초기화하면 공식 PostgreSQL 이미지에서 해당 사용자는 초기 관리자 역할로 생성됨.
일반 애플리케이션 운영에서는 별도의 최소권한 계정을 추가하는 것이 더 안전함.
```

```sql
CREATE USER appuser WITH PASSWORD '강력한비밀번호';
GRANT CONNECT ON DATABASE wizdb TO appuser;
```

20. 자주 쓰는 Docker 명령

목적	명령
설정 검사	docker compose config

목적	명령
시작	docker compose up -d
상태	docker ps
중지	docker compose stop
재시작	docker compose restart
컨테이너 제거	docker compose down
로그	docker logs wiz-postgresql
실시간 로그	docker logs -f wiz-postgresql
DB 접속	docker exec -it wiz-postgresql psql -U devuser -d wizdb

21. 데이터 영구저장 원리

```text
Rocky Linux
/home/devuser/docker/postgresql/data
                 │
                 │ Bind Mount
                 ▼
Docker PostgreSQL
/var/lib/postgresql/data
```

```bash
docker compose down으로 컨테이너를 제거해도 호스트의 data 폴더는 남음. 다시 docker compose up -d 하면 기존 DB를 사용함.
```

주의:

```text
~/docker/postgresql/data 디렉터리는 실제 DB 원본이므로 함부로 삭제하면 안 됨.
```

22. 백업과 복구 기초

특정 DB를 SQL 파일로 백업하는 기본 예임.

```bash
docker exec wiz-postgresql \
  pg_dump -U devuser -d wizdb > ~/wizdb_postgresql_backup.sql
```

복구 전에는 대상 DB와 권한을 확인해야 하며, 운영 환경에서는 정기 백업과 별도 저장소 보관을 권장함.

23. 초보자가 기억할 핵심 명령 7개

```bash
cd ~/docker/postgresql
docker compose config
docker compose up -d
docker ps
docker logs wiz-postgresql
docker exec -it wiz-postgresql psql -U devuser -d wizdb
docker compose down
```

24. 최종 점검표

점검 항목	정상 기준
Compose	docker compose config 오류 없음
컨테이너	wiz-postgresql 실행 중
영구저장	~/docker/postgresql/data 사용
DB	wizdb 존재
사용자	devuser 접속 가능
외부 포트	5432 LISTEN
pg_hba.conf	192.168.0.0/24 허용
방화벽	192.168.0.0/24 → 5432 허용
Windows 테스트	TcpTestSucceeded : True
GUI	DBeaver 연결 가능